

Igny CLI User Guide

The Crucible workbench command line, beta edition

Igny is the command-line interface to the Crucible workbench. It installs EDA tools, keeps each project's tools in a named environment, and runs them without touching your system PATH. This guide covers the supported path: create an environment and a workspace, install and run tools, then lock the result so a teammate can reproduce it exactly.

TO DO THIS	RUN	SECTION
Create a toolbox	<code>igny env create <name></code>	05
Bind a project to it	<code>igny workspace create --env <name></code>	06
Install a tool	<code>igny tool install <tool></code>	08
Run it	<code>igny run <binary></code>	10
Make it reproducible	<code>igny workspace lock</code>	12

CONTENTS

What This Guide Covers

Nineteen sections, in the order you meet them. Sections 01 to 03 are enough to get a tool running; the rest fill in the lifecycle around it.

01	Core Concepts	03
02	Check Your Installation	05
03	Quick Start	06
04	Where Crucible Keeps Things	08
05	Environments	09
06	Workspaces	14
07	Workspace Status	16
08	Installing and Inspecting Tools	17
09	Installing a Compiler by Target	21
10	Running Tools	23
11	Running Scripts	25
12	Lock and Reproduce a Workspace	26
13	Discovering Tools	27
14	Diagnostics	30
15	Registry Cache	30
16	Environment Variables Reference	31
17	Troubleshooting	32
18	End-to-End Recipes	34
19	Inspecting Your Installed CLI	36

What this guide covers

The **stable, supported surface** of the beta CLI. Commands hidden from `--help` are not covered. The `crucible.lock` format is **not yet stable across CLI versions**, so regenerate the lock after upgrading rather than assuming it carries forward.

SECTION 01

Core Concepts

Igny has four concepts, and every command in this guide follows from them. Two of them cause most of the early confusion: the split between a tool being **installed** on the machine and **bound** to an environment, and the difference between the environment your workspace selects and the environment that is currently active.

THE FOUR CONCEPTS

CONCEPT	WHAT IT IS	YOU SEE IT WITH
Catalog tool	A tool Crucible knows how to get	<code>igny catalog</code>
Installed tool	A tool present on this machine	<code>igny tool list</code>
Environment	A named, isolated set of tool bindings	<code>igny env ...</code>
Workspace	A project directory bound to one environment	<code>igny workspace ...</code>

THE NORMAL LIFECYCLE

```
# 1 create or pick an environment
# 2 create a workspace bound to it
# 3 install or sync tools into it
# 4 lock the tool state
# 5 run tools, scripts, or flows
```

Installed on the machine, bound to an environment

The payload (the simulator binary, the toolchain) is downloaded **once** into a shared machine inventory. Each environment then holds a lightweight **binding** to that payload. Installing the same tool into a second environment does not download it again. This is also why uninstall is careful: it removes the machine payload only when no environment still references it.

SECTION 01 · CONTINUED

Selected and Active Environments

The **selected** environment is the one recorded in your workspace's `crucible.toml`. It is a per-project preference. Set it with `igny workspace bind` (pointer only) or `igny env use --workspace`.

The **active** environment is machine-global runtime state: whatever is switched on right now. Set it with `igny env activate` or `igny env use`.

Most of the time the two line up. When `igny run` has to pick an environment, it resolves in this order and stops at the first one that answers:

RESOLUTION ORDER

ORDER	SOURCE
1	The <code>CRUCIBLE_ENV</code> environment variable
2	The workspace-selected environment (<code>crucible.toml</code>)
3	The last activated environment
4	The managed default workspace's environment

SECTION 02

Check Your Installation

Two commands confirm the CLI is on your PATH and show you what it exposes.

PRINT THE CLI VERSION

```
igny --version  
  
0.0.7
```

`igny -v` is a short alias for the same thing.

SHOW THE TOP-LEVEL HELP

```
igny --help
```

The main command groups you will use:

```
igny env ...      manage environments (toolboxes)  
igny workspace ... manage project directories  
igny tool ...     install / inspect tools  
igny compiler ... install compilers by target triplet  
igny run ...      run a bound binary or a script  
igny catalog      browse the supported tool catalog  
igny status       show where you are  
igny doctor       run diagnostics  
igny cache ...    manage the registry cache
```

SECTION 03

Quick Start

Five commands take you from nothing to a running tool. Each one prints a line confirming what it did, so if the sequence stops you can see where.

THE WHOLE SEQUENCE

```
igny env create mysim
igny workspace create --path ./chip_design --env mysim
cd ./chip_design
igny tool install gtkwave
igny run gtkwave
```

STEP BY STEP

Create an environment, which is your project's toolbox. Creating it does **not** activate it:

```
igny env create mysim

Environment 'mysim' created (stack=default)
```

Create a workspace and bind it to that environment:

```
igny workspace create --path ./chip_design --env mysim

Workspace 'chip_design' created (env=mysim)
```

SECTION 03 · CONTINUED

Quick Start: Install and Run

Move into the workspace. Tool commands run inside it bind to its environment:

```
cd ./chip_design
```

Install a tool into the workspace environment:

```
igny tool install gtkwave  
  
Installed gtkwave@3.3.116  
Source: artifact  
Path: /home/you/Crucible/tools/gtkwave/3.3.116  
Bound to env: mysim
```

Run it:

```
igny run gtkwave
```

IF THE WORKSPACE CAME FIRST

If a workspace already exists but points at the wrong environment, or at one that was never created, create the environment and re-point the workspace:

```
cd ./chip_design  
igny env create mysim  
igny workspace bind mysim  
igny tool install gtkwave  
igny run gtkwave
```

SECTION 04

Where Crucible Keeps Things

Tool payloads live under one visible location, which you can move with an environment variable. Your project directory stays yours.

PLATFORM ROOT

Tool payloads live under `~/Crucible/` by default:

```
~/Crucible/  
  tools/  
    <tool-name>/<version>/
```

Override it with `CRUCIBLE_ROOT`:

```
export CRUCIBLE_ROOT=/opt/crucible
```

Your project tree stays clean

Crucible keeps its own configuration and run logs **outside your project tree**. The one file it adds to your project is `crucible.toml`, plus `crucible.lock` once you explicitly lock.

SECTION 05

Environments

An environment is a named, isolated toolbox. Tools you install while a workspace is selected to an environment get **bound** to that environment. This section covers the whole lifecycle: create, list, inspect, activate, rename, move between machines, and delete.

CREATE

```
igny env create sim
# optional stack label
igny env create sim --stack default

Environment 'sim' created (stack=default)
```

Creating an environment does **not** activate it. Do that explicitly with `igny env activate` or `igny env use`.

NAMING RULES

Shared by environments and workspaces: start with a lowercase letter; use only lowercase letters, digits, -, and _; no trailing hyphen; no consecutive hyphens; **63 characters at most**. Underscores are unrestricted, so `my_env` and `tmp_act_1` are both fine. A bad name fails before anything is written to disk.

SECTION 05 · CONTINUED

Listing and Inspecting

LIST

```
igny env list

* default (stack=default) [0 tools]
  sim (stack=default) [4 tools]
~ fw-build (stack=default) [2 tools]
```

Markers: * is the last activated environment, ~ is another active environment, and no marker means inactive. [N tools] is how many tools are bound.

INSPECT ONE ENVIRONMENT

```
igny env show sim

Environment: sim
Stack:      default
Status:     current (active)
Flags:      deletable
Tools:      2
  - verilator 5.020 [artifact]
  - cocotb    1.9.2 [pip]
Workspaces: 1
  - /home/you/projects/uart-soc
```

Status is current (active) when the cursor points here, active when it is on but not the cursor, or inactive. The reserved default environment shows reserved (not deletable).

SECTION 05 · CONTINUED

Activate, Deactivate, Use

```
igny env activate sim
Activated environment 'sim'

igny env deactivate sim
Deactivated environment 'sim'
```

Deactivating is **idempotent**. Deactivating something already inactive still exits 0, and it never deletes the environment or its bindings.

Tier limits on active environments

On the community tier **only one environment can be active at a time**. If you activate a second one, Igny asks whether to deactivate the current one first. In a script or CI there is no terminal to answer the prompt, so pass `--yes` to confirm automatically, or deactivate the other environment before you start.

ACTIVATE AND POINT THE WORKSPACE AT IT

```
# activate only
igny env use sim
# activate AND set it as the workspace's selected env
igny env use sim --workspace
# only update the workspace pointer
igny env use sim --no-activate --workspace

Activated environment 'sim'
Workspace 'uart-soc' now uses environment 'sim'
```

To change the pointer without activating anything, `igny workspace bind` does the same job (Section 06).

SECTION 05 · CONTINUED

The Environment Shell

`igny env shell` drops you into an interactive subshell where every bound tool is on your PATH and every bound Python library is importable. It works like activating a `virtualenv`, for your whole toolbox.

```
# a named environment
igny env shell sim
# the resolved environment for the current context
igny env shell

$ igny env shell sim
Entering Crucible env shell – bound tools active. Type 'exit' to leave.
$ verilator --version
Verilator 5.020 2024-01-01
$ python -c "import cocotb; print(cocotb.__version__)"
1.9.2
$ exit
Left Crucible env shell.
```

The subshell sets `$IGNY_ENV` to the environment name. Leaving it restores your original shell exactly, so **nothing global is changed**, and the subshell's exit code becomes the command's exit code.

SECTION 05 · CONTINUED

Rename, Move, Delete

RENAME

```
igny env rename sim sky130-sim
```

```
Environment 'sim' renamed to 'sky130-sim'.
```

```
Warning: workspace 'uart-soc' is still bound to 'sim'.
```

```
Run: igny workspace bind sky130-sim --path /home/you/projects/uart-soc
```

The directory, metadata, and active state are updated atomically. Workspaces bound to the old name are **not auto-updated**; the warning gives the exact repair command for each one, and is omitted when nothing references the old name.

EXPORT / IMPORT

```
igny env export sim ./sim-env
```

```
Exported environment 'sim' to ./sim-env.zip
```

```
igny env import ./sim-env.zip
```

```
Imported environment 'sim'
```

The archive's filename stem becomes the imported environment name. Importing over an existing name fails, and neither command restores active runtime state.

DELETE

```
igny env delete sim
```

```
Deleted environment 'sim'
```

Deletion is **refused**, with a non-zero exit, for a missing environment, the currently active environment, the reserved default, or one still referenced by a workspace.

SECTION 06

Workspaces

A workspace is any project directory containing a `crucible.toml`. That file records the project's identity and the environment it selects. Workspace creation is **marker-based**: the file is the artifact that matters, so a directory without one is not a workspace no matter what it contains.

CREATE

```
# named, in a new directory
igny workspace create my-chip --path ./my-chip --env sim
# in the current directory
igny workspace create --env sim
cd my-chip

Workspace 'my-chip' created (env=sim)
```

If `--env` is omitted, the active environment is used. If the name is omitted, the directory name becomes the workspace name. `igny workspace create` **refuses to overwrite** an existing workspace, and exits non-zero if `--env` names an environment that does not exist.

INSPECT

```
igny workspace show

Name       : my-chip
Type       : Custom
Path       : /home/you/projects/my-chip
ID         : 7f3a9c21
Environment : sim
Known Envs : sim, fw-build
Default Flow: -
Lock File  : Missing
```

Type is Custom for your projects and Managed for the system default workspace. Lock File reads Present once you run `igny workspace lock`.

LIST

```
igny workspace list

guardian-asic  /home/you/projects/guardian-asic
my-chip        /home/you/projects/my-chip
```

The list is global to your install. On a fresh machine it prints `No workspaces registered.` and still exits 0.

SECTION 06 · CONTINUED

Bind, Rename, Unregister

BIND TO A DIFFERENT ENVIRONMENT

A pure pointer update. It changes the workspace's selected environment without activating it:

```
ign workspace bind sim
Workspace 'my-chip' bound to environment 'sim'
```

Use `ign env use sim --workspace` to activate and bind in one step.

RENAME

Run from inside the workspace:

```
ign workspace rename my-chip-v2
Workspace 'my-chip' renamed to 'my-chip-v2'
```

The **workspace ID stays stable** across a rename. Only the display name changes, in both `crucible.toml` and the registry.

DELETE (UNREGISTER)

```
ign workspace delete --yes
ign workspace delete --path ./my-chip --yes

Workspace 'my-chip' unregistered from Crucible tracking.
Project files preserved: /home/you/projects/my-chip
```

This removes the workspace from Crucible's registry only: **your project files stay on disk**, and so does `crucible.toml`. Omit `--yes` to be prompted. The managed default workspace cannot be deleted.

SECTION 07

Workspace Status

`ign status` is **read-only**. It never creates or modifies anything, so it is safe to run anywhere, including on a machine where Crucible has never been configured.

```
ign status
# inspect a directory without cd-ing into it
ign status --path ./my-chip
```

INSIDE A WORKSPACE

```
Workspace : uart-soc
Path      : /home/you/projects/uart-soc
Env       : sim [active]
Tools     : 3
Lock file : present
```

OUTSIDE ANY WORKSPACE

```
Workspace : none
```

The [active] or [inactive] tag shows whether the workspace's bound environment is currently in the active set.

SECTION 08

Installing and Inspecting Tools

Where you run an install decides what it binds to. Inside a workspace, a tool binds to that workspace's **selected environment**. Outside one, it binds to the managed default environment.

HOW INSTALLS ARE BOUND

WHERE YOU RUN IT	WHAT HAPPENS
Inside a workspace	Machine inventory + the workspace's selected environment
Outside a workspace	Machine inventory + the managed default environment

ACQUISITION FAMILIES

SOURCE	MEANING
pip	A Python package in Crucible's isolated tool directory
artifact	A prebuilt archive, downloaded and extracted
docker	A Docker image, pulled or referenced

Crucible picks the family per tool. Artifact installs **verify size and checksum metadata** when available, and repair a stale cached archive by downloading it once more before failing.

For pip-backed tools that expose a command, Crucible writes a self-contained launcher next to the install, so the tool runs through `igny run`, inside `igny env shell`, or directly, and the package stays importable.

SECTION 08 · CONTINUED

Install, List, Check

INSTALL

```
igny tool install verilator
# a specific version
igny tool install fusesoc --version 2.4.5

Installed verilator@5.020
Source: artifact
Path: /home/you/Crucible/tools/verilator/5.020
Bound to env: sim
```

If it is already present, the install reports `verilator@5.020` is already installed.

LIST INSTALLED TOOLS

```
igny tool list

Installed tools:
  verilator@5.020 (artifact)
  fusesoc@2.4.5 (pip)
```

On an empty machine it prints `No tools installed`.

CHECK WHETHER A TOOL IS INSTALLED

```
igny tool check verilator
igny tool check fusesoc --version 2.4.5

verilator is installed
fusesoc@2.4.5 is installed
```

It **exits non-zero** when the tool or the requested version is missing, so it works as a guard in scripts.

SECTION 08 · CONTINUED

Inspecting a Tool in Depth

`ign tool show` gives a **four-part view** of everything Crucible knows about a tool, or about a single binary that a tool provides:

```
ign tool show verilator
# vvp is provided by iverilog
ign tool show vvp
# resolve against a specific env
ign tool show verilator --env fw-build
# machine-readable
ign tool show verilator --json
```

THE FOUR BLOCKS

BLOCK	WHAT IT TELLS YOU
Resolved for context	What <code>ign run <name></code> would execute now, and if it can
Catalog	Domains, sources, and the binaries the tool provides
Installed on machine	Which versions are installed
Environment bindings	Which environments bind a package providing it

Reach for this when `ign run` says a binary is not bound. Ask about a companion binary such as `vvp` and the catalog block names its provider and the install hint, `ign tool install iverilog`.

SECTION 08 · CONTINUED

Unbind and Uninstall

UNBIND: KEEP THE PAYLOAD, DROP THE BINDING

```
igny tool unbind fusesoc
```

This removes the tool's binding from the current environment but leaves the machine-installed payload, and any other environments' bindings, intact.

UNINSTALL

```
igny tool uninstall fusesoc 2.4.5  
# version derived when only one is bound  
igny tool uninstall fusesoc  
  
Uninstalled fusesoc@2.4.5
```

Uninstall is **forgiving**. It first unbinds the tool from your current environment, then removes the machine payload only if no other environment still references it. If another environment still uses it, you get a notice rather than an error:

```
Unbound fusesoc from 'sim'. Payload retained – still referenced by env(s): fw-build
```

It exits non-zero only for genuine problems: the version is not installed, the version is ambiguous and you did not specify one, or files cannot be removed.

SECTION 09

Installing a Compiler by Target

The `igny compiler` group is a lens over the tool catalog. You name a target triplet such as `riscv32` and Igny resolves it to the catalog tool that provides it. Afterwards it is an **ordinary tool**: it lists, binds, and runs like any other.

LIST AVAILABLE COMPILERS

```
igny compiler list
```

One row per toolchain: its target name, whether it is installed, and the binaries it provides.

SHOW A TARGET

```
igny compiler show riscv32  
Resolved target 'riscv32' → riscv32-gcc
```

When the target resolves to a differently named tool, Igny prints that note first, followed by the same four-block view as `igny tool show`.

RESOLUTION ORDER

ORDER	MATCH	EXAMPLE
1	Exact catalog tool name	<code>riscv32-gcc</code>
2	Exact provided-binary name	<code>riscv32-buildroot-linux-musl-gcc</code>
3	Prefix match	<code>riscv32 → riscv32-gcc</code>

Resolution runs within `Compiler-domain` tools only.

SECTION 09 · CONTINUED

Install, Uninstall, Ambiguity

INSTALL

```
ignpy compiler install riscv32
# into a specific env
ignpy compiler install riscv-none-elf --env fw-build
ignpy compiler install riscv32 --version 14.2.0
```

After install, every binary the toolchain ships is runnable directly:

```
ignpy run riscv32-buildroot-linux-musl-gcc --version
```

UNINSTALL

```
ignpy compiler uninstall riscv32
ignpy compiler uninstall riscv-none-elf --env fw-build
```

This unbinds the toolchain from the resolved environment and removes it from the machine once nothing else references it, exactly like `ignpy tool uninstall`.

AMBIGUOUS TARGETS

If a bare target matches more than one toolchain, Igny **refuses to guess** and lists the candidates:

```
Target 'riscv' is ambiguous. Matching compilers:
- riscv32-gcc
- riscv64-gcc
- riscv-none-elf-gcc
Specify a more precise target.
```

SECTION 10

Running Tools

`igny run` executes any binary the resolved environment provides. Everything after the binary name reaches the tool untouched, and `igny run` exits with the **tool's own exit code**, so it drops into a Makefile or CI step without a wrapper.

```
igny run verilator --version
```

If the tool fails, Igny leaves its output visible, adds a `Run failed:` summary, and prints a `Full` output: pointer to the captured run log when one exists. On POSIX systems, signal-killed tools use the usual shell `128+N` exit status.

OVERRIDE THE ENVIRONMENT FOR ONE RUN

```
igny run --env sim verilator --version
```

`--env` is recognised **only before the binary name**, so a tool with its own `--env` flag still receives it: `igny run mytool --env prod` passes `--env prod` to `mytool`.

MULTI-BINARY PACKAGES

If `iverilog` declares `provides = ["iverilog", "vvp"]`, both names work without a separate `vvp` binding:

```
igny run iverilog -o sim.vvp tb.v
igny run vvp sim.vvp
igny run yosys -p "synth -top top; write_json out.json" rtl.v
```

SECTION 10 · CONTINUED

Where You Run It, and When It Fails

Interactive tools attach to your current terminal. Terminal tools such as yosys and renode show their normal prompt or banner; GUI tools such as gtkwave launch normally.

WHERE YOU RUN IT

Run `igny run` from **anywhere inside your workspace**, including a subdirectory. Crucible finds the workspace by walking up from your current directory. The tool launches in the folder you are standing in, so relative input and output paths resolve against your current directory.

CHOOSING THE ENVIRONMENT

Besides `--env`, you can set `CRUCIBLE_ENV` for the run:

```
CRUCIBLE_ENV=fw-build igny run riscv-none-elf-gcc --version
```

WHEN A BINARY ISN'T BOUND

```
# names the provider and the install hint
igny tool show vvp
igny tool install iverilog
```

RESERVED-NAME ESCAPE HATCH

If a binary's name collides with an `igny run` sub-command such as `script` or `tool`, disambiguate with `--binary`:

```
igny run --binary script my_arg
```

SECTION 11

Running Scripts

`ign run script` executes a script with the environment's context applied: its tools on PATH and its libraries importable. Output streams to your terminal and is **captured to a log** at the same time.

Run it from anywhere inside your workspace. The script launches in the folder you are standing in, so relative paths resolve against your current directory.

```
ign run script ./scripts/build.py
```

```
R=8.20 kOhm  
PASS opamp_square_wave_generator  
Script completed successfully
```

PASSING ARGUMENTS

Arguments for the script go after `--`:

```
ign run script ./scripts/build.py -- --target sim --jobs 4
```

A failing script reports its exit code and exits non-zero: `Script failed (exit code 2)`.

LOGS

Add `--log` to print the log path on success; it is **always shown when a script fails**. Use `--quiet` to keep the terminal clean, and pair it with `--log` to capture to the log only.

```
ign run script ./scripts/build.py --log
```

SECTION 12

Lock and Reproduce a Workspace

This is how a project becomes reproducible for a teammate or for CI. The **lock file** records what was actually bound, and `sync` rebuilds it on another machine.

WRITE THE LOCK

```
ign workspace lock

Locked workspace 'my-chip' (env='sim') - 3 tool(s) -> crucible.lock
```

`crucible.lock` captures each bound tool's **name, version, source, paths, and checksum or URL** where available. Commit `crucible.toml` and `crucible.lock` together.

REPRODUCE ON ANOTHER MACHINE

```
git clone <project-repo>
cd <project>
ign workspace sync

Synced 3 tool(s) to environment 'sim': verilator, cocotb, fusesoc
```

Re-running it is **idempotent** and reports `Already bound: verilator, cocotb, fusesoc`. The managed default workspace cannot be synced.

The lock file is the reproduction source of truth

`sync` reads `crucible.lock`, not `crucible.toml`. If the lock names an environment that does not exist on this machine, `sync` **creates** it; if `crucible.toml` points elsewhere, `sync` **repoints** the workspace to the lock's environment. There is no override flag, because an override would build a different toolchain from the one the lock recorded.

SECTION 13

Discovering Tools

The catalog lists everything Crucible knows how to acquire, not only what is already installed.

Design suites group those tools into named bundles for a task, so you can pull a whole capability without remembering each tool by hand.

BROWSE THE CATALOG

```
# compact view, grouped by domain
igny catalog
# names, install state, domains, descriptions
igny catalog --view full
igny catalog --tag simulation
# 'vp' and 'simulation' expand to the same cluster
igny catalog --tag vp
```

--tag is repeatable to combine filters. The bundled RTL catalog includes verible for linting and formatting and sv2v for SystemVerilog-to-Verilog translation, alongside iverilog, verilator, and yosys.

DESIGN SUITES

Shipped suites include sim-basic, a minimal Icarus Verilog and Verilator setup, and riscv-suite: a JVM build stack with temurin-jdk and mill, bare-metal riscv-none-elf-gcc, and Verilator and GTKWave for simulation, with optional Linux toolchains, renode, and synthesis/FPGA tools.

```
igny suite list
# bundled tools, required vs optional, pinned versions
igny suite show sim-basic
```

SECTION 13 · CONTINUED

Acquiring and Installing Suites

A suite has two verbs. `acquire` downloads the tools onto your machine. `install` also makes them usable with `igny run` in your current environment.

```
# required tools only
igny suite acquire sim-basic
# add one optional tool, or every optional tool
igny suite acquire sim-basic --with verilator
igny suite acquire sim-basic --all-optional

igny suite install sim-basic
igny suite install sim-basic --with verilator
igny suite install sim-basic --all-optional
# into a named environment
igny suite install sim-basic --env lab
```

Required tools are always included; optional ones are opted in with `--with` (repeatable) or `--all-optional`. Every tool is attempted and you get a per-tool result plus a summary. If any tool fails, the command **exits non-zero**. If the environment already holds a suite tool at a different version, `install` leaves it and reports a conflict.

acquire or install

Use `acquire` when you want the payload on the machine now and will decide later which environment uses it. Use `install` when you want the suite ready to run where you are working. `install` reuses whatever `acquire` downloaded, so running both does not download anything twice.

SECTION 13 · CONTINUED

Binding an Already-Acquired Tool

Acquired tools show up in `igny tool list`. To make one usable in the current environment **without downloading anything**, bind it:

```
igny tool bind <name>
```

`igny tool bind` is the inventory-only counterpart to `igny tool install`. `igny tool install <name>` also works, and acquires the tool first if it is not on the machine yet.

WHICH COMMAND TO USE

YOU WANT	COMMAND	DOWNLOADS
Tools on the machine	<code>igny suite acquire</code>	If missing
An acquired tool usable here	<code>igny tool bind</code>	Never
One tool, ready to run	<code>igny tool install</code>	If missing
A whole suite, ready to run	<code>igny suite install</code>	If missing

SECTION 14

Diagnostics

`ign doctor` checks the runtime Crucible depends on (Python, Git, Docker, registry readiness) and reports what it finds **without raw tracebacks**. Run it first when something fails in a way the error message does not explain.

HUMAN-READABLE

```
ign doctor
```

MACHINE-READABLE, FOR SCRIPTS AND CI

```
ign doctor --json  
  
{  
  "ok": true,  
  "exit_code": 0,  
  "checks": []  
}
```

Both forms **exit non-zero** when a critical check fails.

REGISTRY CACHE

Crucible caches the **Ayudha** tool registry locally. `ign cache sync` refreshes it; `ign cache clear` removes cached registry metadata **and** downloaded artifact archives. Neither needs a workspace.

```
ign cache sync  
Registry synced - 42 tools available  
  
ign cache clear  
Cache cleared
```

If there was nothing to clear, it prints `Cache is already empty`.

SECTION 16

Environment Variables Reference

Five variables change where Crucible puts things and which environment it resolves. Keep CRUCIBLE_ROOT and IGNITION_HOME pointed at **separate directories**.

VARIABLE	PURPOSE
CRUCIBLE_ROOT	Visible platform root for tool payloads (~/.Crucible/)
IGNITION_HOME	Where Crucible keeps config, registry cache, and logs
CRUCIBLE_ENV	Forces the env igny run / igny env shell resolve to
CRUCIBLE_VERSION	Version reported by igny --version (else dev)
\$IGNY_ENV	Set by igny env shell in the subshell (read-only)

EXAMPLE

```
# tool payloads and internal state in separate directories
export CRUCIBLE_ROOT=/opt/crucible
export IGNITION_HOME=/opt/ignition-home
```

SECTION 17

Troubleshooting

These are the failures that come up most often in beta use. Each one names the cause, because the fix rarely makes sense without it.

A BINARY ISN'T BOUND

`igny run <tool>` says the binary isn't bound: it is not installed in, or bound to, the resolved environment.

```
# names the provider and the install hint
igny tool show <binary>
igny tool install <provider>
```

THE WORKSPACE ISN'T FOUND

`igny run` walks up from your current directory to find the workspace, so run it from inside or at your workspace. Check where you are with `igny status`.

A TOOL RUNS IN ONE ENVIRONMENT BUT NOT ANOTHER

Installing a payload **does not bind it everywhere**. Install it from inside the second environment's workspace:

```
# see which envs bind it
igny tool show <tool>
# from inside the target workspace
igny tool install <tool>
```

SECTION 17 · CONTINUED

Troubleshooting, Continued

ACTIVATE REFUSES A SECOND ENVIRONMENT

On the community tier **one environment is active at a time**. Deactivate the current one, or let activation do it:

```
igny env deactivate <current>
# or
igny env activate <new> --yes
```

A WORKSPACE STILL SHOWS A RENAMED ENVIRONMENT

Renames do not patch workspace pointers. Run the repair command the rename warning printed:

```
igny workspace bind <new-name> --path <workspace-path>
```

UNINSTALL KEPT THE PAYLOAD

Expected when another environment still binds the tool; it is a **notice, not an error**. Unbind it from the other environments first if you want it gone.

DELETE IS REFUSED

You cannot delete the active environment, the reserved default, or one a workspace still references. Deactivate or re-bind, then delete.

NETWORK OR REGISTRY ERRORS

During install or `igny cache sync`: confirm connectivity, then run `igny doctor`. It flags registry readiness along with other runtime issues.

SECTION 18

End-to-End Recipes

Four sequences, each a complete working session starting from an empty directory. Read them to see how the pieces fit together before you adapt one to your own project.

RECIPE A — A FRESH SIMULATION PROJECT

```
ign y env create sim
ign y workspace create uart-soc --path ./uart-soc --env sim
cd uart-soc

ign y tool install verilator
ign y tool install gtkwave
ign y workspace lock

ign y run verilator --version
ign y status
```

RECIPE B — A FIRMWARE BUILD WITH A CROSS-COMPILER

```
ign y env create fw-build
ign y workspace create blinky --path ./blinky --env fw-build
cd blinky

ign y compiler install riscv-none-elf
ign y run riscv-none-elf-gcc -O2 -c main.c -o main.o
ign y workspace lock
```

SECTION 18 · CONTINUED

Recipes C and D

RECIPE C — REPRODUCE A TEAMMATE'S PROJECT

```
git clone <project-repo>
cd <project>

# installs and binds everything from crucible.lock
igny workspace sync
# confirm tool count and lock state
igny status
igny run verilator --version
```

RECIPE D — WORK IN AN INTERACTIVE SHELL

```
cd uart-soc
igny env shell

# inside the subshell, all bound tools are on PATH
verilator --version
python -c "import cocotb; print(cocotb.__version__)"
exit
```

Recipes A and C are the two halves of one workflow: **lock** what you built, then **sync** what a teammate cloned.

SECTION 19

Inspecting Your Installed CLI

This guide covers the stable, supported path. Your **installed build** is the authority on what it actually exposes.

```
igny --help
igny env --help
igny workspace --help
igny tool --help
igny compiler --help
igny run --help
```

Each subgroup's `--help` lists its current options and is **always authoritative** for the version you have installed. If it disagrees with this guide, trust the CLI and tell us so the guide can be corrected.

CONTACT

Product information

<https://www.ignytion.io>

Support and questions about the beta

info@ignytion.io
